

UCI Opinion Review 데이터셋을
활용한 클러스터링 및 문서 유사도
분석은 정답 레이블이 존재하지 않는
비지도 학습 문제로, 데이터의 잠재적
구조를 탐색하는 데 중점을 두었다.
머신 러닝이 예측 뿐 아니라 데이터
이해와 구조 분석에도 효과적으로
활용될 수 있음을 확인하였다.

4 장

Opinion Review 분석

제 4 장 목차

4. Opinion Review 분석 프로젝트	1
4.1 문제 정의 및 데이터 이해	1
4.1.1 프로젝트 배경 및 목표.....	1
4.1.2 해결하고자 하는 문제 및 평가 지표.....	3
4.1.3 데이터 설명 및 구조	7
4.2 기초 설정 및 데이터 전처리.....	8
4.2.1 실험 환경 설계 및 PyCaret 전용 Conda 환경 구성	8
4.2.2 필수 라이브러리 및 모듈 로드	9
4.2.3 데이터 로드 및 텍스트 전처리 전략.....	10
4.2.4 벡터화 전략: TF-IDF vs Count 기반 표현	13
4.2.5 시각화 및 실험 환경 설정.....	13
4.2.6 실험 파이프라인 요약.....	14
4.2.7 재현성 및 팀 협업 관점에서의 설계 의의	14
4.2.8 요약.....	14
4.3 토픽 모델링	16
4.3.1 LDA (Latent Dirichlet Allocation) 기반 토픽 모델링	16
4.3.2 NMF (Non-Negative Matrix Factorization) 기반 토픽 모델링	20
4.4 클러스터링(Clustering) 분석 및 최적화	25
4.4.1 클러스터링 알고리즘 성능 비교 및 선택.....	25

4.4.2 최적 클러스터 수 결정 시각화	27
4.5 유사도 분석 및 시각화	30
4.5.1 LDA 기반 유사도 분석 및 시각화	30
4.5.2 상세 유사도 분석 및 결과 저장	35
4.5.3 클러스터 유사도 시각적 결과	39
4.6 결론 및 제언	41
4.6.1 주요 결과 요약	41
4.6.3 개선 방안 및 향후 계획	45
4.6.4 실습 정리 및 요약	46
4.7 참고자료 및 부록	48
4.7.1 참고 자료	48
4.7.2 부록	49

4. Opinion Review 분석 프로젝트

4.1 문제 정의 및 데이터 이해

4.1.1 프로젝트 배경 및 목표

4.1.1.1 Opinion Review 데이터셋 개요

본 프로젝트는 UCI Machine Learning Repository 에서 제공하는 *Opinion Review* 데이터셋을 활용하여, 문서 군집화(Document Clustering) 및 문서 간 유사도 분석(Document Similarity Analysis)을 수행하는 것을 목표로 한다.

Opinion Review 데이터셋은 총 51 개의 텍스트 문서 파일로 구성되어 있으며, 각 문서는 다음과 같은 온라인 리뷰 플랫폼에서 수집된 실제 사용자 리뷰를 포함한다.

- TripAdvisor: 호텔 리뷰
- Edmunds.com: 자동차 리뷰
- Amazon.com: 전자제품 리뷰

각 문서는 평균적으로 약 100 개 내외의 문장으로 구성되어 있으며, 파일명 자체가 리뷰 주제(도메인 및 대상)를 나타내는 암묵적 레이블 역할을 한다. 본 데이터는 아래 UCI 공식 경로를 통해 다운로드하여 사용하였다.

- <https://archive.ics.uci.edu/dataset/191/opinosis+opinion+frasl+review>

본 데이터셋은 사전 정의된 정답 라벨이 없는 비지도 학습(Unsupervised Learning) 환경의 텍스트 데이터로, 문서 간 의미적 유사성과 잠재 주제 구조를 모델이 스스로 학습해야 한다는 특징을 가진다.

4.1.1.2 프로젝트 목표

본 프로젝트의 주요 목표는 다음과 같다.

- 1) 텍스트 전처리 및 벡터화 기법 비교

- 단순 토큰화 및 불용어 제거부터 확장된 전처리 전략 적용
- TF-IDF, Count 기반 벡터라이징 등 다양한 표현 방식 비교

2) 문서 군집화 알고리즘 성능 비교

- K-Means 기반 군집화 실습 결과를 출발점으로
- 다양한 군집화 알고리즘(Affinity Propagation, DBSCAN, OPTICS 등) 적용

3) PyCaret 기반 AutoML 활용

- `pycaret.clustering` 모듈을 활용하여 여러 군집화 모델을 자동 비교
- 내부 평가 지표를 기준으로 Base Model 및 Best Model 선정

4) 비지도 학습 환경에서의 모델 평가 방법 이해

- 라벨이 없는 상황에서 군집 품질을 어떻게 판단하는지에 대한 실험적 이해

4.1.1.3 기대 효과

본 프로젝트를 통해 다음과 같은 학습 효과를 기대한다.

- 텍스트 데이터의 특성에 따른 전처리 전략의 중요성 체득
- 문서 벡터화 방식이 군집 결과에 미치는 영향 이해
- 비지도 학습 환경에서 사용되는 군집 평가 지표의 의미와 한계 이해
- PyCaret 기반 AutoML 을 활용한 실험 자동화 및 모델 비교 경험 습득

4.1.1.4 관련 파일 목록

- Opinion Review 원본 텍스트 파일 (총 51 개)

- 전처리 유틸리티 모듈 (`utils/preprocessing`)
- 군집화 실험 및 평가 코드
- 결과 시각화 이미지 및 로그 파일

4.1.2 해결하고자 하는 문제 및 평가 지표

본 프로젝트에서 해결하고자 하는 핵심 문제는 다음과 같다.

정답 라벨이 존재하지 않는 리뷰 문서 집합에서, 문서 간 의미적 유사성을 기반으로 의미 있는 군집 구조와 잠재 주제를 자동으로 도출할 수 있는가?

Opinion Review 데이터셋은 문서 단위의 클래스 레이블을 제공하지 않는 비지도 학습(Unsupervised Learning) 환경의 데이터이다. 따라서 일반적인 지도 학습에서 사용되는 Accuracy, Precision, Recall, F1-score 와 같은 지표를 직접적으로 적용할 수 없다.

이러한 특성으로 인해, 본 프로젝트에서는 문서 간 거리, 군집 내부 응집도, 군집 간 분리도를 기반으로 한 내부 평가 지표(Internal Validation Metrics)를 사용하여 군집화 결과의 품질을 평가하였다.

4.1.2.1 비지도 학습 vs 지도 학습 평가 지표 비교

지도 학습과 비지도 학습은 문제 정의와 평가 방식에서 근본적인 차이를 가진다.

구분	비지도 학습 (Unsupervised)	지도 학습 (Supervised)
라벨 존재 여부	없음	있음
평가 기준	데이터 내부 구조	예측값 vs 정답
대표 지표	Silhouette, CH, DB, ARI, NMI	Accuracy, Precision, Recall, F1, ROC-AUC
평가 방식	군집 응집도·분리도 기반	분류 정확도 기반
한계	절대적 기준 부재	라벨 품질 의존

[표 4.1.1 평가 지표 비교]

비지도 학습에서는 “정답을 얼마나 맞췄는가”가 아니라, “데이터가 얼마나 자연스럽게 뭉였는가”가 평가의 핵심이 된다.

4.1.2.2 주요 평가 지표

본 프로젝트에서는 이후 모든 군집화 및 토픽 모델링 결과를 동일한 기준에서 비교·해석하기 위해 다음과 같은 내부 평가 지표를 공통 기준으로 사용하였다.

1) Silhouette Score (실루엣 점수)

각 문서가 자신의 클러스터 내부에서 얼마나 잘 응집되어 있는지와 다른 클러스터와 얼마나 명확히 분리되는지를 동시에 평가할 수 있는 지표로, 군집 구조의 직관적인 품질 판단에 적합하다.

- 범위: -1 ~ 1
- 값이 클수록 우수
- 각 데이터 포인트가
- 자신의 클러스터 내부에서는 얼마나 잘 응집되어 있고
- 다른 클러스터와는 얼마나 잘 분리되어 있는지를 측정
- 1 에 가까움: 군집 분리 우수
- 0 근처: 군집 간 중첩
- 음수: 잘못된 군집 할당

2) Calinski-Harabasz Index (CH 지수)

클러스터 간 분산 대비 클러스터 내 분산 비율을 기반으로 하여, 군집 간 거리와 내부 밀집도를 정량적으로 비교할 수 있으며, 여러 알고리즘 간 상대 비교에 유리하다.

- 값이 클수록 우수

- 클러스터 간 분산 대비 클러스터 내 분산 비율을 평가
- 군집 간 거리가 멀고 내부가 조밀할수록 높은 값
- 계산 속도가 빨라 비교 실험에 적합

3) Davies-Bouldin Index (DB 지수)

각 클러스터 간 유사도의 평균을 측정함으로써, 군집 간 중첩 여부를 평가하며, 값이 낮을수록 군집 분리가 우수함을 의미한다.

- 값이 낮을수록 우수
- 각 클러스터 간 유사도를 기반으로 군집 품질을 평가
- 클러스터 간 겹침이 적고 분리가 명확할수록 낮은 값
- 0 에 가까울수록 이상적

4) Clusters (클러스터 수)

- 알고리즘이 최종적으로 생성한 클러스터 개수
- K-Means 계열은 사전 지정
- DBSCAN, OPTICS 등은 데이터 특성에 따라 자동 결정

5) N_Noise (노이즈 포인트 수)

- DBSCAN, OPTICS 등 밀도 기반 알고리즘에서
- 어떤 클러스터에도 속하지 못한 데이터 수
- 값이 클수록 군집화 품질 저하 가능성

6) Time (s)

- 알고리즘 실행 시간(초)
- 실무 적용 시 확장성과 효율성 판단 지표로 활용

4.1.2.3 PyCaret 결과 해석 가이드

PyCaret의 `clustering` 모듈은 여러 군집화 알고리즘을 동일한 데이터와 지표 기준으로 비교할 수 있도록 지원한다.

그러나 단일 지표(Silhouette Score 등)만을 기준으로 모델을 선택할 경우,

- 과도하게 많은 클러스터 생성
- 의미 없는 단일 클러스터 형성
- 노이즈 포인트 과다 발생

등의 문제가 발생할 수 있다.

따라서 본 프로젝트에서는 Silhouette Score, CH Index, DB Index, 클러스터 수, 실행 시간을 종합적으로 고려하여 이후 장(4.3~4.5)에서 사용할 기반 모델을 선정하였다.

4.1.2.4 결과 해석 가이드

- Affinity Propagation
- Silhouette Score 가 상대적으로 가장 높음
- 다수의 클러스터 생성 → 세분화된 군집 구조
- KMeans, Spectral, Birch
- Silhouette Score 가 낮아 군집 간 분리도가 약함
- DBSCAN, MeanShift

- 단일 클러스터만 생성 → 의미 있는 군집 형성 실패
- K-Modes
- 음수 Silhouette Score → 군집 구조 부적합
- OPTICS
- 낮은 Silhouette와 높은 노이즈 비율 → 군집 품질 제한적

요약하면, 비지도 학습에서는 예측값과 정답을 직접 비교할 수 없으므로, 데이터 구조 자체를 평가하는 내부 지표를 통해 군집 품질을 판단해야 한다.

4.1.3 데이터 설명 및 구조

4.1.3.1 데이터 규모

- 총 문서 수: 51 개
- 문서 길이: 평균 약 100 문장
- 데이터 유형: 비정형 텍스트 데이터

4.1.3.2 핵심 변수

- 리뷰 텍스트 본문
- 파일명(암묵적 주제 정보)
- 벡터화 후 생성되는 고차원 특성 벡터

4.2 기초 설정 및 데이터 전처리

본 절에서는 4.1 절에서 정의한 실험 목표와 평가 기준을 실제로 구현하기 위해, 본 프로젝트에서 사용한 실험 환경 구성 방식, 텍스트 전처리 전략, 그리고 벡터화 및 AutoML 실험을 위한 기초 설정을 설명한다.

특히 본 프로젝트는

- 여러 군집화 알고리즘을 동일 조건에서 반복 비교하고
- PyCaret 기반 AutoML 을 활용하여 실험을 자동화하는 것을 핵심 목표로 설정하였기 때문에,

실험 환경의 재현성(reproducibility)과 팀 단위 협업을 고려한 환경 통일성을 중요한 설계 요소로 두었다. 특히 PyCaret 기반 AutoML 을 활용함으로써, 동일한 전처리·평가 기준 하에서 다수의 군집화 알고리즘을 체계적으로 비교·선택할 수 있는 실험 구조를 구현하고자 하였다.

4.2.1 실험 환경 설계 및 PyCaret 전용 Conda 환경 구성

4.2.1.1 PyCaret 전용 환경 분리의 필요성

PyCaret 은 내부적으로 `scikit-learn`, `lightgbm`, `xgboost`, `matplotlib`, `numpy` 등 다수의 핵심 라이브러리에 의존하는 AutoML 프레임워크이다.

기존 머신러닝 개발 환경에 PyCaret 을 직접 설치할 경우,

- 라이브러리 버전 충돌
- 기존 실습 코드 실행 오류
- 팀원별 실행 환경 차이로 인한 결과 재현성 저하

와 같은 문제가 발생할 수 있다.

이에 본 프로젝트에서는 PyCaret 전용 Conda 환경을 별도로 구성하여 실험 환경을 완전히 분리하였다.

4.2.1.2 Conda 환경 구성 및 팀 공유 방식

실험 환경의 일관성을 확보하기 위해, PyCaret 전용 환경 설정 파일을 다음과 같이 구성하였다.

- ``/config/pycaret_env.yml``
 - PyCaret 실행에 필요한 Python 및 라이브러리 버전 명시
- ``/config/pycaret_setup.md``
 - Conda 환경 생성, 활성화, 설치 및 테스트 절차 문서화

해당 파일들은 프로젝트 저장소의 ``config`` 디렉토리 하위에 두고, 팀원 전원이 동일한 환경을 생성하도록 공유하였다.

이와 같은 방식은 다음과 같은 장점을 가진다.

- 팀원 간 실험 결과의 재현성 확보
- AutoML 실험 시 발생 가능한 환경 이슈 최소화
- 이후 추가 실험 또는 재실행 시 환경 재구성 비용 절감

환경 설정 파일의 상세 내용은 부록(별첨)으로 제공한다.

4.2.2 필수 라이브러리 및 모듈 로드

본 프로젝트에서 사용한 주요 라이브러리는 다음과 같다.

- ``scikit-learn``
- 텍스트 벡터화 및 기본 군집화 알고리즘 활용

- `TfidfVectorizer`, `CountVectorizer`
- 문서 벡터 표현 방식 비교
- `pycaret.clustering`
- 군집화 알고리즘 자동 비교 및 평가
- 사용자 정의 전처리 모듈 (`utils/preprocessing`)
- 텍스트 정제 로직의 일관성 확보

특히 PyCaret의 `clustering` 모듈은 여러 군집화 알고리즘을 동일 데이터·동일 지표 기준으로 비교할 수 있도록 지원하여, 4.1 절에서 정의한 평가 지표(Silhouette, CH, DB 등)를 실험 단계에서 일관되게 적용하는 데 핵심적인 역할을 하였다.

4.2.3 데이터 로드 및 텍스트 전처리 전략

4.2.3.1 데이터 로드 방식 및 입력 구조

Opinion Review 원본 데이터는 `/data` 디렉토리에 저장된 다수의 `.data` 파일로 구성되며, 본 프로젝트에서는 `utils.preprocessing.load\_file\_data()` 함수를 통해 해당 파일들을 일괄 로드하였다.

각 파일은 다음과 같은 구조로 데이터프레임에 적재된다.

- `filename`: 파일명(확장자 제외) — 문서의 암묵적 주제 정보
- `opinion\_text`: 파일 내용을 `opinion\_text` 컬럼에 적재하여 문서 단위 데이터프레임 형태로 통합하였다.

또한 원본 데이터의 인코딩 이슈를 최소화하기 위해 파일 읽기 시 `latin1` 인코딩을 적용하여, 다양한 출처(TripAdvisor/Edmunds/Amazon)에서 수집된 텍스트의 특수문자 처리 안정성을 확보하였다.

4.2.3.2 전처리 로직의 표준화: TextPreprocessor 설계

Opinion Review 데이터는 비정형 텍스트로 구성되어 있으므로, 군집화 이전에 텍스트 노이즈를 제거하고 표현을 정규화하는 과정이 필수적이다.

본 프로젝트의 전처리는 `TextPreprocessor` 클래스로 캡슐화하여, 모든 실험에서 동일한 규칙이 적용되도록 표준화하였다. 전처리의 목적은 군집화/토픽모델링 단계에서 문서 간 거리 계산이 안정적으로 수행되도록 노이즈를 제거하고 표현을 정규화하는 데 있다. 주요 전처리 단계는 다음과 같다.

- HTML 태그 제거: BeautifulSoup 기반 텍스트 추출(`remove\_html=True`)
- URL 제거: 정규표현식 기반 http(s), www 패턴 제거(`remove\_urls=True`)
- 이메일 제거: 이메일 패턴 제거(`remove\_emails=True`)
- 숫자 제거: 숫자 문자열 제거(`remove\_numbers=True`)
- 소문자 변환: 대소문자 변동으로 인한 희소성 증가 방지(`lowercase=True`)
- 구두점/특수문자 제거: `[^\w\Wws]` 패턴을 공백으로 치환(`remove\_punctuation=True`)
- 공백 정규화: 연속 공백을 단일 공백으로 축약 후 strip 처리
- 토큰화(Tokenization) : 기본적으로 NLTK `word\_tokenize` 사용(예외 시 split fallback)
- 불용어 제거(Stopwords): NLTK English stopwords 기반 제거(`remove\_stopwords=True`)
- 최소 토큰 길이 필터링: `min\_word\_length=2` 미만 토큰 제거
- 표제어 추출(Lemmatization): 기본값으로 WordNetLemmatizer 적용(`use\_lemmatization=True`)

(옵션으로 Stemming 도 제공하나 본 프로젝트 기본 설정은 Lemmatization 중심)

이와 같은 전처리 파이프라인은 `TextPreprocessor.preprocess_text()`에서 단일 텍스트에 적용되며, `preprocess_dataframe()`을 통해 데이터프레임 전체에 일괄 적용된다. 전처리 결과는 `processed_text` 컬럼으로 저장하여, 이후 벡터화(TF-IDF/Count)와 PyCaret 기반 군집화 비교 실험이 동일 입력을 사용하도록 구성하였다. 이러한 전처리 수준은 단순한 텍스트 정제가 아니라, 문서 간 거리 계산과 군집 품질 평가의 안정성을 확보하기 위한 최소한의 정규화 단계로 설계되었다.

4.2.3.3 품질 관리: 빈 문서 제거 및 전처리 통계 기록

전처리 이후에는 내용이 비어버린 문서(Empty document)를 자동으로 제거하여 군집화 품질 저하를 방지하였다. 구현상 `processed_text`가 공백/빈 문자열인 문서를 마스킹하여 제거하고 인덱스를 재정렬한다.

또한 전처리 과정에서 다음과 같은 통계 정보를 수집하여(클래스 내부 `stats`) 데이터 품질을 점검할 수 있도록 설계하였다.

- 원본 문서 수(`original_docs`)
- 전처리 후 빈 문서로 제거된 수(`empty_after_preprocessing`)
- 전처리 후 평균 단어 수(`avg_words_after`)

이 통계는 실험 반복 과정에서 “전처리 옵션 변경 → 데이터 유효 문서 수 변화 → 군집 성능 변화”를 추적하는 근거로 활용할 수 있어, AutoML 기반 반복 실험(다양한 알고리즘 비교)의 신뢰성을 높이는 장치로 기능하며, PyCaret 기반 AutoML 환경에서 알고리즘 변경에 따른 성능 차이가 전처리 편차가 아닌 모델 특성에서 기인했음을 보장하는 근거로 활용된다.

4.2.3.4 기본 전처리 실행 함수 및 확장 가능성

본 프로젝트에서는 `get_default_data()` 함수를 통해 데이터 로드 + 기본 전처리를 일괄 수행하도록 구성하였다. 기본 설정은 HTML/URL/이메일/숫자/구두점 제거, 소문자 변환,

불용어 제거, Lemmatization 을 포함하며, 필요 시 사용자 정의 불용어(`get_custom_stopwords`)를 추가하여 도메인 특화 정제도 확장 가능하도록 설계하였다.

4.2.4 벡터화 전략: TF-IDF vs Count 기반 표현

본 프로젝트에서는 전처리 이후 문서 표현 방식에 따라 군집 결과가 어떻게 달라지는지를 비교하기 위해, 두 가지 벡터화 방식을 병행하여 사용하였다.

- TF-IDF Vectorizer
- 단어의 문서 내 중요도를 반영
- K-Means 및 일반 군집화 실험에 주로 활용
- CountVectorizer
- 단어 출현 빈도 기반 표현
- LDA 기반 토픽 모델링(4.3 절) 단계에서 활용

이는 4.1 절에서 설정한 “전처리 및 벡터화 기법 비교”라는 프로젝트 목표를 실험적으로 검증하기 위한 설계이며, 이후 장에서 벡터화 방식에 따른 군집 및 토픽 모델링 결과 차이를 비교 분석한다.

4.2.5 시각화 및 실험 환경 설정

군집 결과 및 평가 지표 시각화를 위해 다음과 같은 환경 설정을 적용하였다.

- 한글 시각화를 위한 Malgun Gothic 폰트 설정
- 그래프 해상도 및 레이아웃 통일
- 실험 결과 이미지 자동 저장 구조 설계

이를 통해 팀원 간 결과 비교 시 시각적 해석의 일관성을 확보하였다.

4.2.6 실험 파이프라인 요약

본 프로젝트의 전체 실험 파이프라인은 다음과 같은 단계로 구성된다.

1. 원본 텍스트 데이터 로드
2. 표준화된 텍스트 전처리 수행
3. TF-IDF/Count 기반 벡터화
4. PyCaret 기반 군집화 알고리즘 자동 비교
5. 내부 평가 지표 기반 모델 선택
6. 결과 시각화 및 해석



[그림 4.2.6 실험 파이프라인]

이 파이프라인은 이후 장(4.3~4.5)에서 수행되는 토픽 모델링, 군집화, 유사도 분석의 공통 기반으로 사용된다.

4.2.7 재현성 및 팀 협업 관점에서의 설계 의의

본 절에서 정의한 환경 구성 및 전처리 전략은 단일 실험을 넘어 반복 가능하고 확장 가능한 분석 구조를 목표로 설계되었다. PyCaret 전용 환경 분리와 설정 파일 공유를 통해 팀원 간 실행 결과의 일관성을 확보하였으며, 표준화된 전처리 모듈을 통해 실험 조건 변경 시에도 비교 가능한 결과를 안정적으로 도출할 수 있었다.

4.2.8 요약

본 절에서는 Opinion Review 데이터 분석을 위한 실험 환경 구성, 전처리 전략, 벡터화 방식 선택을 설명하였다.

특히 PyCaret 전용 Conda 환경을 분리·공유함으로써 AutoML 기반 대규모 비교 실험을 안정적으로 수행할 수 있는 기반을 마련하였으며, 이후 4.3~4.5 절에서 수행되는 토픽 모델링, 군집화, 유사도 분석은 본 절에서 정의한 환경과 전처리 설정을 공통적으로 사용한다.

본 절에서 정의한 실험 환경과 전처리·벡터화 설정은 이후 모든 분석 단계의 공통 기반으로 사용된다.

다음 장(4.3)에서는 동일한 전처리 결과를 바탕으로 LDA 및 NMF 기반 토픽 모델링을 수행하여,

Opinion Review 문서 집합 내에 잠재된 주제 구조를 식별하고 각 문서의 의미적 분포를 정량적으로 분석한다.

특히 벡터화 방식(Count 기반 표현)이 토픽 모델링 결과에 미치는 영향을 함께 비교·해석한다.

4.3 토픽 모델링

4.3.1 LDA (Latent Dirichlet Allocation) 기반 토픽 모델링

4.3.1.1 LDA 모델 개요

LDA 는 통계 기반의 비지도 학습 토픽 모델링 알고리즘으로, 문서 집합에서 잠재적인 토픽을 자동으로 발견하였다. 각 문서는 여러 토픽의 혼합으로, 각 토픽은 여러 단어의 확률 분포로 표현된다.

핵심 가정:

- 각 문서는 여러 토픽의 혼합(mixture)으로 구성
- 각 토픽은 특정 단어들의 확률 분포
- 문서 생성 과정을 확률적으로 모델링

4.3.1.2 데이터 전처리

전처리 파이프라인

```
# TextPreprocessor 클래스를 사용한 체계적 전처리
preprocessor = TextPreprocessor(
    remove_html=True,      # HTML 태그 제거
    remove_urls=True,      # URL 제거
    remove_numbers=True,   # 숫자 제거
    lowercase=True,        # 소문자 변환
    remove_stopwords=True, # 불용어 제거
    use_lemmatization=True  # 표제어 추출
)
```

전처리 결과

- 원본 문서 수: 51 개
- 제거된 빈 문서: 0 개
- 최종 문서 수: 51 개
- 평균 단어 수: 1,267.3 개

전처리 단계를 통해 HTML 태그, URL, 숫자 등의 노이즈를 제거하고, 영어 불용어를 제거한 후 표제어 추출(Lemmatization)을 적용하여 단어를 기본형으로 통일하였다.

4.3.1.3 LDA 모델 구축

문서-단어 행렬(DTM) 생성

1) Scikit-learn 의 CountVectorizer 를 사용하여 DTM 을 생성했습니다:

```
vectorizer = CountVectorizer(
    max_df=0.8,          # 80% 이상 문서에 등장하는 단어 제외
    min_df=2,            # 최소 2 개 문서에 등장해야 포함
    stop_words="english", # 영어 불용어 제거
    token_pattern="[a-zA-ZW-][a-zA-ZW-]{2,}" # 최소 3 글자 이상
)
```

파라미터 설명:

- max_df=0.8: 너무 빈번한 일반적 단어 제외
- min_df=2: 희귀 단어 제외로 노이즈 감소
- token_pattern: 최소 3 글자 제한으로 의미 있는 단어만 선택

2) LDA 모델 학습

```
lda_model = LatentDirichletAllocation(
    n_components=6,          # 토픽 개수
    random_state=23,         # 재현성 보장
    max_iter=50,             # 최대 반복 횟수
    learning_method="online", # 온라인 학습
    n_jobs=-1                 # 병렬 처리
)
```

하이퍼파라미터 설정:

- 토픽 개수(n_components): 6 개
- 51 개 문서에 대해 적절한 추상화 수준

- 너무 적으면 일반화, 너무 많으면 과적합 위험
- 학습 방법: Online 학습으로 대규모 데이터 효율적 처리
- 병렬 처리: 모든 CPU 코어 활용(-1)

3) 모델 성능 지표

- Perplexity: 700.4577
- Perplexity 는 낮을수록 좋은 지표
- 모델이 새로운 문서를 얼마나 잘 예측하는지 측정
- 약 700 의 값은 중간 수준의 성능을 나타냄

4) 토픽 분석 결과

- 추출된 6 개 토픽과 주요 키워드

토픽 번호	주요 키워드 (상위 5 개)	해석
토픽 1	location, battery, hotel, life, price	위치/배터리/가격 관련 - 호텔 위치, 배터리 수명, 가격 대비 가치
토픽 2	room, mileage, comfortable, interior, seat	실내 공간/편안함 - 객실/차량 내부, 좌석 편안함, 연비
토픽 3	video, direction, sound, voice, speed	미디어/네비게이션 - 비디오 품질, 길 안내, 음성/음향
토픽 4	staff, room, service, hotel, food	호텔 서비스 - 직원 서비스, 객실, 음식
토픽 5	screen, keyboard, button, size, page	전자기기 인터페이스 - 화면, 키보드, 버튼, 크기
토픽 6	free, wine, coffee, morning, hotel	호텔 편의시설 - 무료 서비스, 조식, 커피

- 토픽 해석

추출된 토픽들은 다음과 같은 제품/서비스 카테고리를 대표합니다:

- 전자제품 성능 (토픽 1, 5): 배터리 수명, 화면 품질, 가격

- 호텔/숙박 (토픽 4, 6): 서비스 품질, 편의시설
- 차량/교통 (토픽 2): 내부 공간, 연비, 좌석
- 네비게이션/멀티미디어 (토픽 3): 음성 안내, 비디오 재생

5) 문서-토픽 분포

● 분포 특성

각 문서는 6 개 토픽에 대한 확률 분포로 표현됩니다.

예시:

문서 1: [0.0004, 0.0004, 0.9979, 0.0004, 0.0004, 0.0004]

→ 토픽 3(네비게이션)에 99.79% 집중

문서 2: [0.0003, 0.9441, 0.0003, 0.0548, 0.0003, 0.0003]

→ 토픽 2(공간/편안함)에 94.41% 집중

문서 5: [0.6162, 0.0001, 0.0053, 0.0001, 0.3783, 0.0001]

→ 토픽 1(61.62%)과 토픽 5(37.83%)의 혼합

● 분포 패턴 분석

단일 토픽 지배형: 대부분 문서가 하나의 토픽에 90% 이상 집중

혼합형: 일부 문서는 2-3 개 토픽의 혼합 (예: 문서 5)

명확한 주제 구분: 토픽 간 경계가 뚜렷하여 해석이 용이

6) 후속 분석 활용

● 특징 벡터로서의 활용

LDA 의 문서-토픽 분포는 다양한 후속 분석의 입력 특징으로 활용 가능합니다:

가능한 활용 방안:

감성 분석: 토픽 분포 → 긍정/부정 예측

제품 분류: 토픽 분포 → 제품 카테고리 예측

추천 시스템: 유사 토픽 분포 문서 추천

클러스터링: 토픽 기반 문서 군집화

4.3.2 NMF (Non-Negative Matrix Factorization) 기반 토픽 모델링

NMF 모델 구축 및 문서별 토픽 분포 추출.

NMF는 비음수 행렬 분해 기법으로, 잠재적인 토픽 구조를 발견하는 또 다른 통계 기반의 토픽 모델링 알고리즘입니다. 행렬 X 를 두 개의 비음수 행렬 W 와 H 의 곱으로 분해합니다 ($X \approx W \times H$).

- X (문서-단어 행렬, DTM): 문서와 단어 간의 빈도 또는 TF-IDF 값.
- W (문서-토픽 행렬): 각 문서가 토픽에 얼마나 기여하는지를 나타냈다.
- H (토픽-단어 행렬): 각 토픽이 단어에 얼마나 기여하는지를 나타냅니다 (토픽의 키워드).

NMF는 주로 TF-IDF 가중치와 함께 사용되어 LDA보다 더 명확하고 해석하기 쉬운 토픽을 추출하는 경향이 있습니다.

4.3.2.1 NMF 모델 구축 및 결과

Scikit-learn의 NMF 모델을 사용하여 5개의 토픽을 추출하였다. NMF는 일반적으로 TF-IDF 기반의 문서-단어 행렬을 입력으로 사용하며, 이는 단어의 중요도에 가중치를 부여하였다.

- 벡터화: TfidfVectorizer 사용
 - `max_df=0.8, min_df=2`, 최소 3글자 이상 단어(`token_pattern="[a-zA-ZW-][a-zA-ZW-]{2,}"`) 사용.

- 모델 하이퍼파라미터:
 - n_components (토픽 개수): 5 개
 - init="nndsvd": 초기화 방법으로 'Non-negative Double Singular Value Decomposition'을 사용하여 학습 속도와 품질을 향상시켰습니다.
- 모델 성능 지표:
 - Reconstruction Error (재구성 오차): 5.8484 (오차가 낮을수록 원본 행렬을 잘 재구성했다는 의미)

4.3.2.2 추출된 5 개 토픽과 주요 키워드

NMF 모델에서 추출된 5 개의 토픽과 상위 5 개 키워드는 다음과 같습니다.

토픽 번호	주요 키워드 (상위 5 개)	해석
토픽 1	room, hotel, clean, bathroom, staff	호텔 시설 및 청결 (객실, 청결도, 욕실)
토픽 2	screen, keyboard, size, direction, speed	전자제품 인터페이스 및 사용성 (화면, 키보드, 크기, 속도/방향)
토픽 3	interior, seat, comfortable, mileage, gas	차량 내부 및 연비 (실내, 좌석 편안함, 연비, 연료)
토픽 4	battery, life, performance, ipod, keyboard	전자제품 성능 및 하드웨어 (배터리 수명, 성능, 특정 기기)
토픽 5	service, food, hotel, location, staff	호텔 서비스 및 편의 (직원 서비스, 음식, 위치)

1) 토픽 해석 요약

NMF 결과는 LDA 결과와 유사하게 전자제품 (토픽 2, 4)과 호텔/숙박 (토픽 1, 5), 그리고 차량/교통 (토픽 3)의 세 가지 주요 카테고리를 명확하게 분리하고 있습니다. 특히 호텔 관련 토픽이 '시설/청결' (토픽 1)과 '서비스/편의' (토픽 5)로 세분화된 것이 특징입니다.

2) 문서-토픽 분포

NMF 모델을 통해 계산된 문서-토픽 분포(W)는 각 문서가 5 개 토픽에 대해 얼마나 기여하는지를 나타냈다.

문서-토픽 분포 (첫 10 개 문서):

[[0.0000 0.1579 0.0112 0.0000 0.0161] → 토픽 2 (전자제품 인터페이스)

[0.3707 0.0000 0.0000 0.0000 0.0000] → 토픽 1 (호텔 시설)

[0.0000 0.0000 0.0000 0.8062 0.0004] → 토픽 4 (전자제품 성능)

[0.0000 0.0000 0.0000 0.8103 0.0000] → 토픽 4 (전자제품 성능)

[0.0000 0.0000 0.0000 0.8427 0.0000] → 토픽 4 (전자제품 성능)

[0.0016 0.1751 0.0020 0.0099 0.0016] → 토픽 2 (전자제품 인터페이스)

[0.0230 0.0000 0.6432 0.0000 0.0000] → 토픽 3 (차량 내부 및 연비)

[0.0489 0.0000 0.6397 0.0000 0.0000] → 토픽 3 (차량 내부 및 연비)

[0.0000 0.2054 0.0163 0.0000 0.0268] → 토픽 2 (전자제품 인터페이스)

[0.0000 0.2652 0.0083 0.0000 0.0098] → 토픽 2 (전자제품 인터페이스)

대부분의 문서가 하나의 토픽에 높은 가중치를 갖거나 (예: 문서 2, 3, 7), 2~3 개의 토픽에 분포하는 (예: 문서 1) 경향을 보였다. 이는 NMF 가 비교적 희소한 (Sparse) 토픽 분포를 생성하여 문서의 주제를 명확하게 구분하는 데 효과적임을 시사하였다.

후속 분석 활용: 문서 유사도 분석

LDA 및 NMF 를 통해 추출된 문서-토픽 분포(W)는 원본 문서의 고차원 단어 공간을 저차원의 토픽 공간으로 압축한 특징 벡터 (Feature Vector) 역할을 하였다. 이 특징 벡터를 사용하여 문서 간의 유사도를 측정할 수 있습니다.

3) 문서 유사도 분석 함수 정의

```
def calculate_document_similarity(doc_topic_dist, metric="cosine"):
    """
    문서-토픽 분포를 기반으로 문서 간의 유사도 행렬 계산
    """
    if metric == "cosine":
        # 코사인 유사도: 두 벡터 간의 각도에 기반, 방향의 유사성 측정 (0~1)
        similarity_matrix = cosine_similarity(doc_topic_dist)
    elif metric == "euclidean":
        # 유클리드 거리: 두 벡터 간의 직선 거리 측정 (거리가 짧을수록 유사)
        # 유사도(Similarity) = 1 / (1 + Distance) 형태로 변환하여 유사도 행렬 생성
        distance_matrix = euclidean_distances(doc_topic_dist)
        similarity_matrix = 1 / (1 + distance_matrix)
    else:
        raise ValueError("Unsupported metric. Choose 'cosine' or 'euclidean'.")
    # DataFrame 으로 변환하여 시각화 및 분석 용이하게 함
    similarity_df = pd.DataFrame(similarity_matrix,
                                index=[f"Doc {i+1}" for i in range(doc_topic_dist.shape[0])],
                                columns=[f"Doc {i+1}" for i in range(doc_topic_dist.shape[0])])
    return similarity_df

def visualize_similarity_heatmap(similarity_df, title="Document Similarity Heatmap"):
    """
    문서 유사도 행렬을 히트맵으로 시각화
    """
    plt.figure(figsize=(12, 10))
    sns.heatmap(similarity_df, cmap="YlGnBu", annot=False, fmt=".2f", linewidths=.5,
                linecolor='black')
    plt.title(title, fontsize=16)
    plt.show()
```

4) 문서 유사도 분석 활용 방안 (NMF 토픽 분포 사용)

NMF의 문서-토픽 분포(doc_topics_sklearn)를 사용하여 코사인 유사도를 계산하고 히트맵으로 시각화할 수 있습니다.

- 동일 주제 문서 식별: 유사도 행렬에서 값이 1에 가까운 문서 쌍은 동일한 제품/서비스에 대한 리뷰이거나, 매우 유사한 측면(예: 둘 다 '배터리 성능'에 대한 이야기)을 다루고 있음을 의미하였다.
- 클러스터링 전처리: 유사도 행렬을 기반으로 문서를 그룹화(클러스터링)하여 토픽 기반의 리뷰 군집을 생성할 수 있습니다.

4.4 클러스터링(Clustering) 분석 및 최적화

4.4.1 클러스터링 알고리즘 성능 비교 및 선택

4.4.1.1 PyCaret 기반 다중 클러스터링 모델 비교 및 최적 알고리즘 선정

본 절에서는 LDA 기반 문서-토픽 분포를 입력하여, PyCaret의 clustering module을 활용한 다중 클러스터링 알고리즘 비교 실험을 수행한다. 이를 위해

perform_advanced_pycaret_clustering() 함수를 구현하여 데이터 전처리, 모델 학습, 클러스터 할당, 성능 평가, 결과 저장까지 일관된 분석 파이프라인을 구성하였다.

비교 대상 알고리즘은 K-Means, Affinity Propagation(AP), MeanShift, Spectral Clustering(SC), Hierarchical Clustering(HCLUST), DBSCAN으로 총 6종이며, 각 알고리즘은 동일한 입력 데이터와 조건에서 학습·

평가되었다. 특히 K-Means, SC, HCLUST의 경우 군집 수를 3으로 고정하였고, AP는 군집 수를 자동으로 결정하도록 설정하였다.

- perform_advanced_pycaret_clustering()함수는 부록에 별첨으로 코드를 붙였다.

4.4.1.2 다양한 클러스터링 기법 평가와 Affinity Propagation 선택 근거

Algorithm	N_Clusters	Silhouette	Davies-Bouldin
AP	5	0.9017	0.2581
SC	3	0.7707	0.4131
HCLUST	3	0.7707	0.4131
KMEANS	3	0.7602	0.5954
MEANSHIFT	3	0.6496	0.7794
DBSCAN	5	0.1669	1.4138

[표 4.4.1]

각 클러스터링 알고리즘의 성능 평가는 Silhouette Score와 Davies-Bouldin Index를 기준으로 수행하였다.

Silhouette Score는 클러스터 내부 응집도와 클러스터 간 분리도를 동시에 반영하는 지표로, 값이 클수록 군집 품질이 우수함을 의미한다. 반면 Davies-Bouldin Index는

클러스터 간 유사도를 기반으로 계산되며, 값이 작을수록 클러스터 간 분리가 잘 이루어졌음을 나타낸다.

[표 4.4.1]에 제시된 실험 결과에 따르면, AP 는 Silhouette Score 는 0.9017 로 비교 대상 알고리즘 중 가장 높은 값을 기록하였으며, Davies-Bouldin Index 또한 0.2581 로 가장 낮은 수치를 보였다.

이는 AP 가 문서 간 토픽 분포 차이를 가장 효과적으로 반영하여 클러스터 간 분리도가 뛰어난 결과를 생성했음을 의미한다. 또한 AP 는 군집 수를 사전에 지정하지 않고도 안정적인 클러스터 구조를 도출하였다는 점에서, 모델 설정 측면에서도 높은 유연성을 보였다. 이러한 정량적 성능 비교 결과를 바탕으로, 본 실험에서 AP 를 최적의 클러스터링 알고리즘으로 선정하였다.

4.4.1.3 LDA 토픽 분포 기반 클러스터링 성능 검증 및 결과 요약

Cluster	대표 토픽	주요 키워드
0	Topic 2	video, price, free, sound, camera, quality
1	Topic 5	mileage, interior, car, gas, comfort
2	Topic 2	video, price, free, sound, camera
3	Topic 3	battery, screen, keyboard, speed
4	Topic 1	room, location, hotel, staff, service

[표 4.4.2]

Affinity Propagation 기반 클러스터링 결과를 분석한 결과, 각 클러스터는 특정 LDA 토픽을 중심으로 구성되었으며, 대표 토픽과 주요 키워드가 명확하게 구분되는 특성을 보였다. 이는 클러스터링 결과가 단순한 수치적 분리가 아닌, 의미적으로 일관된 주제 구조를 반영하고 있음을 시사한다.

실제로 [표 4.4.2]에 나타난 바와 같이, 호텔 서비스, 자동차 관련 리뷰, 전자기기 사용 경험 등 주제적으로 유사한 문서들이 동일 클러스터로 묶였으며, 이를 통해 LDA 토픽

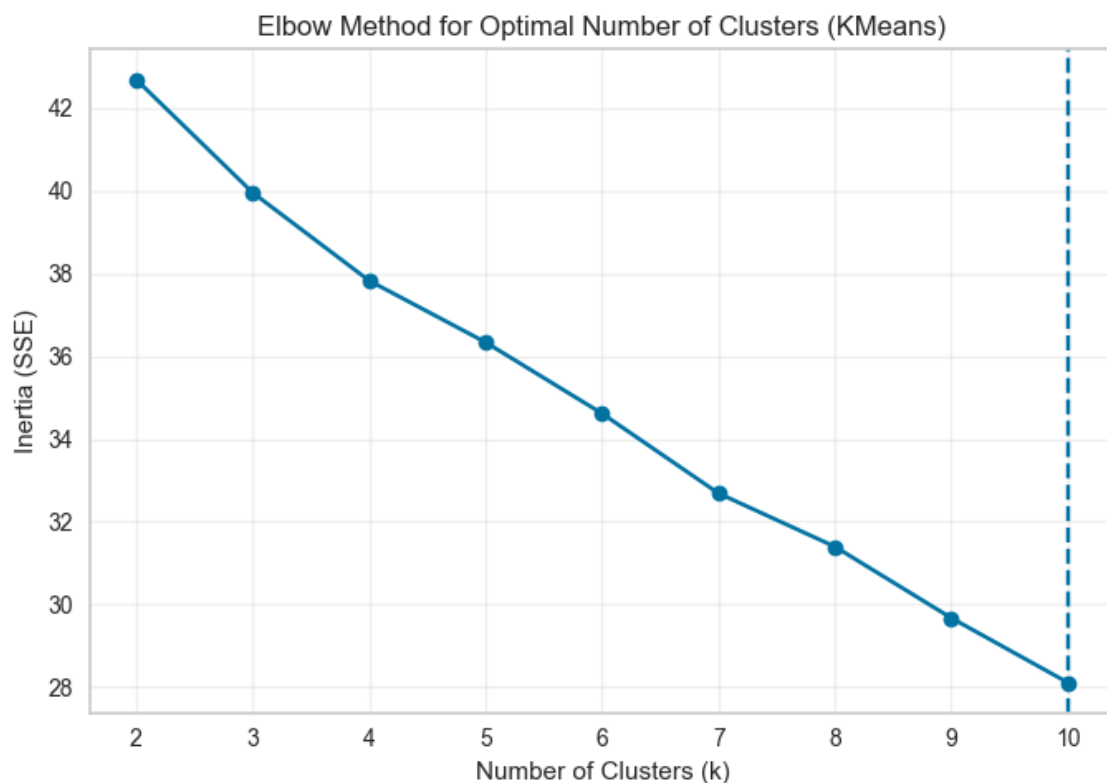
분포가 클러스터링 입력 특징으로서 효과적으로 작동함을 확인하였다. 또한 본 연구에서는 클러스터별 대표 토픽, 토픽 점수, 주요 키워드를 문서 단위로 매핑하여 CSV 파일로 저장함으로써, 결과 해석의 용이성과 후속 분석을 위한 재현성을 확보하였다. 종합적으로, LDA 기반 문서-토픽 분포와 Affinity Propagation의 결합은 비지도 문서 군집화 문제에서 높은 군집 품질과 해석 가능성을 동시에 만족하는 효과적인 접근법임을 검증하였다.

* 각 알고리즘별 CSV는 부록에 넣었다.

4.4.2 최적 클러스터 수 결정 시각화

본 절에서는 KMeans 알고리즘을 기준으로 Elbow Method와 Silhouette Score를 활용하여 최적 클러스터 수를 탐색하였다.

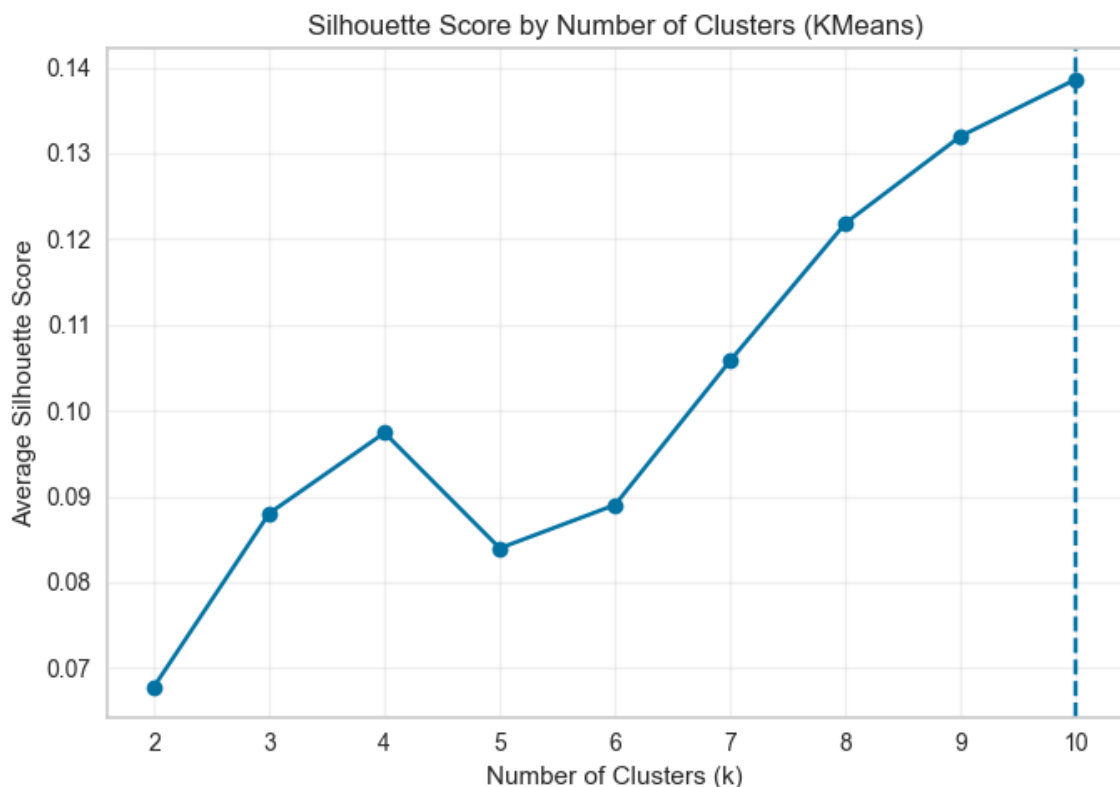
4.4.2.1 엘보우·실루엣 기반 최적 클러스터 수 탐색



[그림 4.4.2 엘보우 방법 (Elbow Method) 시각화]

Elbow Method 분석 그림 4.4.2 는 k 를 2 부터 10 까지 변화시키며 계산한 Inertia(Sum of Squared Errors) 값을 나타낸 Elbow 곡선이다. Inertia 는 클러스터 내부의 응집도를 의미하며, 값이 작을수록 각 클러스터가 더 조밀하게 형성되었음을 의미한다. 그래프를 살펴보면, k 증가에 따라 Inertia 값이 전반적으로 감소하는 추세를 보이나, 특정 지점에서 급격한 감소 후 완만해지는 뚜렷한 엘보우(elbow) 지점은 명확하게 관찰되지 않는다. 이는 TF-IDF 기반 고차원 텍스트 공간에서 문서 분포가 연속적으로 분산되어 있어, 클러스터 수 증가에 따라 응집도 개선이 점진적으로 이루어졌기 때문으로 해석할 수 있다.

따라서 Elbow Method 단독으로는 최적 클러스터 수를 하나의 값으로 명확히 결정하기 어렵다고 판단하였으며, 추가적인 판단 지표로 Silhouette Score 분석을 병행하였다.



[그림 4.4.3 실루엣 점수 (Silhouette Score) 시각화]

Silhouette Score 분석 그림 4.4.3 은 동일한 클러스터 수 범위에 대한 평균 Silhouette Score 의 변화를 시각화한 결과이다. Silhouette Score 는 각 데이터 포인트가 자신이

속한 클러스터에 얼마나 잘 속해 있는지를 나타내는 지표로, 값이 클수록 클러스터 간 분리도와 군집 구조의 타당성이 높음을 의미한다.

분석 결과, $k=2$ 에서 비교적 낮은 Silhouette Score 를 보인 이후, 클러스터 수가 증가함에 따라 전반적으로 점진적인 상승추세가 관찰되었다. 특히 $k \geq 7$ 구간부터 실루엣 점수가 안정적으로 증가하며, $k=10$ 에서 가장 높은 Silhouette Score 가 확인되었다. 이는 클러스터 수 증가에 따라 문서 간 의미적 분리가 보다 명확해졌음을 시사한다.

4.4.2.2 종합적 판단 및 최종 클러스터 수 결정

Elbow Method 에서는 명확한 단일 최적점이 관찰되지 않았으나, Silhouette Score 분석 결과 클러스터 수 증가에 따라 군집 품질이 지속적으로 개선되는 경향을 확인할 수 있었다. 이에 따라 본 연구에서는 Elbow Method 를 통해 후보 범위를 설정하고, Silhouette Score 를 주요 판단 기준으로 활용하여 최적 클러스터 수를 결정하였다. 종합적으로 고려한 결과, Silhouette Score 가 가장 높은 값을 기록한 $k=10$ 을 최적 클러스터 수로 설정하였으며, 이후 분석에서는 해당 기준을 바탕으로 클러스터링 및 후속 유사도 분석을 수행하였다.

4.5 유사도 분석 및 시각화

4.5.1 LDA 기반 유사도 분석 및 시각화

Opinosis 데이터에 대해 먼저 Scikit-learn LDA 기반 토픽 모델링을 수행한 뒤, “문서-토픽 분포”를 활용해서 문서 유사도와 군집 구조를 시각화/분석했다.

- LDA 학습 및 문서-토픽 분포 생성

CountVectorizer 로 전처리된 문장들에서 문서-단어 행렬(DTM)을 생성한 뒤, LatentDirichletAllocation(n_components = n_topics)으로 LDA 모델을 학습했다. 학습된 모델의 components_로 토픽-단어 분포, transform(dtm)으로 각 문서에 대한 문서-토픽 분포(doc_topics_sklearn)를 얻었다. display_topics() 함수로 각 토픽의 상위 키워드를 출력해, 토픽 해석 가능성을 확인했다.

```
=====
주요 토픽 및 키워드 (SKLEARN)
=====
```

```
토픽 1:
room, location, hotel, staff, service
```

```
토픽 2:
video, price, free, sound, camera
```

```
토픽 3:
battery, screen, life, button, keyboard
```

```
토픽 4:
free, location, battery, life, room
```

```
토픽 5:
mileage, interior, car, gas, comfortable
```

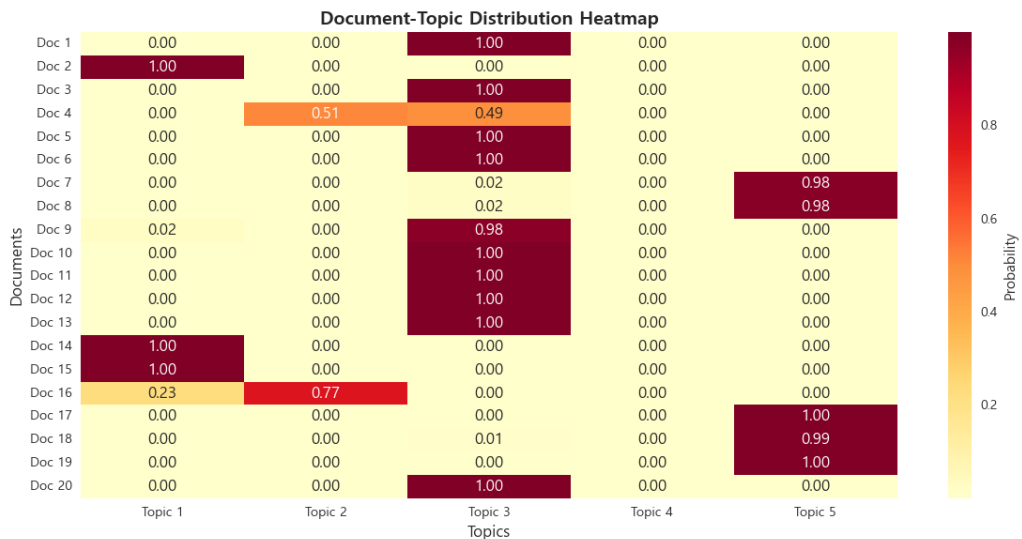
```
문서-토픽 분포 (첫 10개 문서):
```

```
[[5.05240546e-04 4.99567374e-04 9.97994766e-01 4.92903138e-04
 5.07523236e-04]
 [9.98624297e-01 3.45564556e-04 3.46546501e-04 3.39224782e-04
 3.44367034e-04]
 [3.11925716e-04 3.19546296e-04 9.98747620e-01 3.08805283e-04
 3.12102993e-04]
 [4.47842150e-04 5.10142094e-01 4.88517357e-01 4.43646636e-04
 4.49060340e-04]
 [8.38648658e-05 8.42757481e-05 9.99666725e-01 8.20794019e-05
 8.30548139e-05]
 [1.74488169e-04 1.74563217e-04 9.99303941e-01 1.71928624e-04
 1.75078593e-04]
 [1.88077551e-04 1.87972465e-04 1.87386638e-02 1.84946948e-04
 9.80700339e-01]
 [2.73953163e-04 2.73349557e-04 1.67472431e-02 2.68932782e-04
 9.82436521e-01]
 [2.41289824e-02 2.73671983e-04 9.75050191e-01 2.69360492e-04
 2.77794246e-04]
 [5.25449847e-04 5.21074839e-04 9.97917209e-01 5.14420163e-04
 5.21845921e-04]]
```

- 토픽 분포 기반 시각화

visualize_topic_distribution()

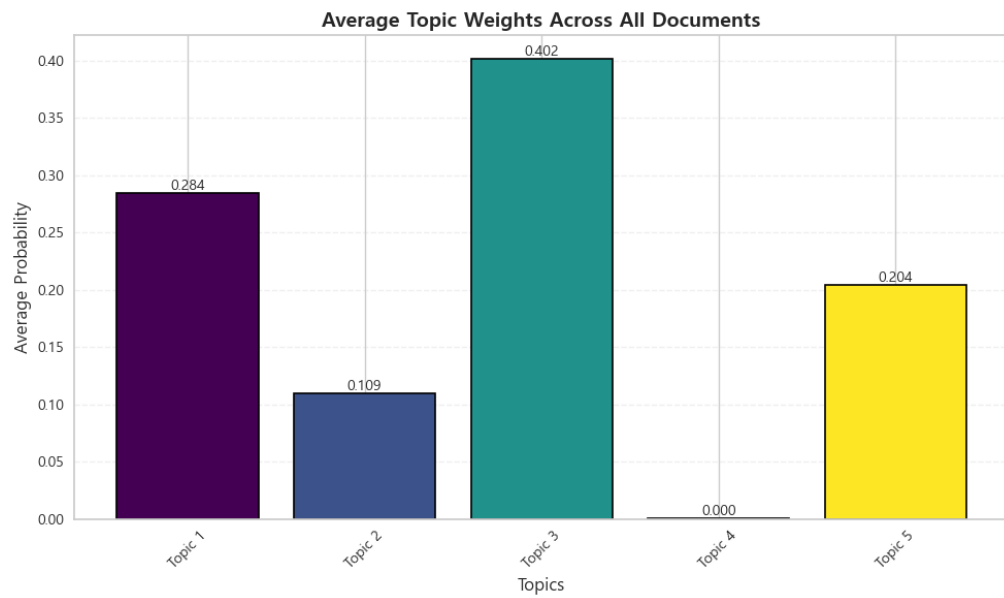
- doc_topics_sklearn 의 상위 N 개 문서를 대상으로, 문서(행) × 토픽(열) 확률 히트맵을 그려 문서마다 어떤 토픽이 지배적인지 시각적으로 확인했다. 이를 통해 특정 토픽에 강하게 몰려 있는 문서와, 여러 토픽이 섞인 문서를 직관적으로 구분할 수 있었다.



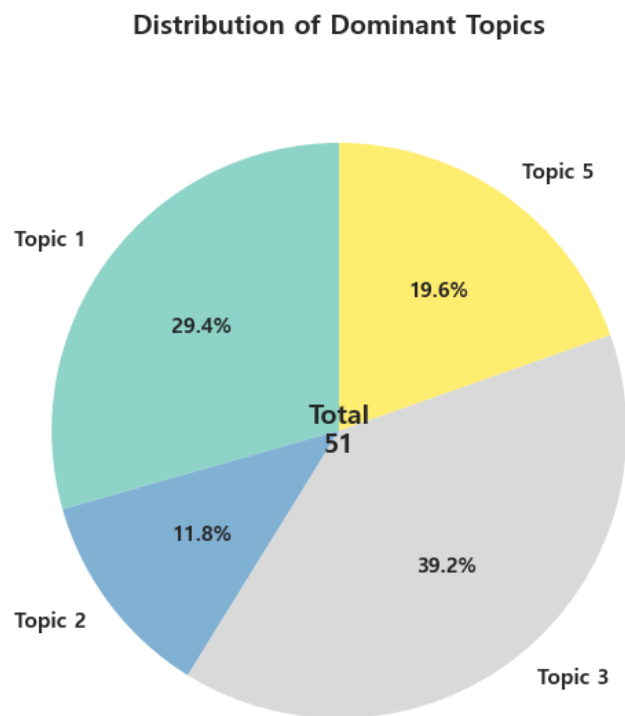
[그림 4.5.1 문서 토픽 분포 히트맵]

visualize_topic_weights() / visualize_dominant_topics()

- 전체 문서에서 각 토픽이 차지하는 평균 비중과, “가장 높은 토픽” 기준으로 문서를 세면 어떤 토픽이 가장 많이 등장하는지를 막대그래프/파이차트로 표현해 Opinosis 코퍼스의 전반적인 주제 분포를 요약했다.



[그림 4.5.2 평균 토픽 비중 차트]

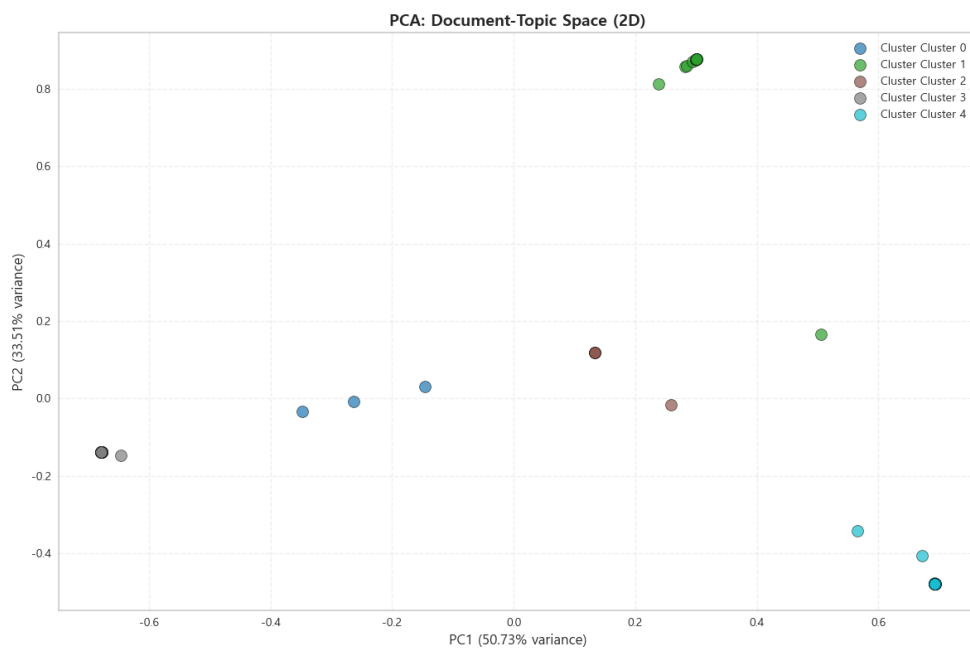


[그림 4.5.3 토픽 등장 비율 차트]

- PCA 기반 2D 토픽 공간 시각화

`visualize_topics_pca()`에서 문서-토픽 분포를 PCA 2차원으로 축소하고, `perform_advanced_pycaret_clustering()`에서 얻은 클러스터 레이블(k-means, AP 등 PyCaret 클러스터링 결과)을 색상으로 구분해 그렸다.

이 플롯을 통해 토픽 분포가 유사한 문서들이 실제로 같은 클러스터에 모여 있는지, 서로 다른 클러스터 간 경계가 얼마나 분리되는지를 직관적으로 검증했다.



[그림 4.5.4 PCA 분포 시각화]

- 문서 유사도 계산 및 유사 문서 탐색

`calculate_document_similarity()` 함수로 문서-토픽 분포에 대한 코사인 유사도 행렬을 계산했다. LDA 결과를 사용했기 때문에 "동일 토픽을 비슷한 비율로 공유하는 문서"일수록 높은 유사도를 가진다.

`find_similar_documents(doc_id, ...)`를 통해 기준 문서(예: 문서 0)와 가장 코사인 유사도가 높은 문서 Top-K를 찾아, 원문 미리보기와 함께 테이블로

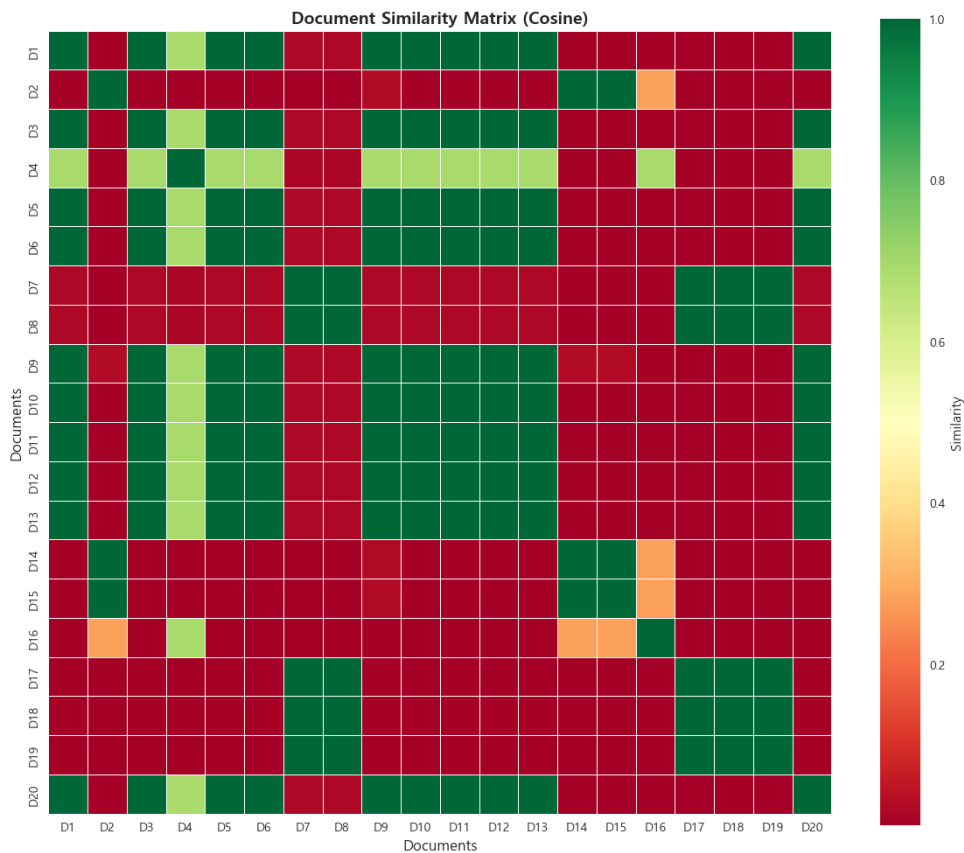
출력함으로서 "같은 토픽 구조를 가지는 문서끼리 실제로 내용도 비슷한지"를 정성적으로 확인할 수 있었다.

- 클러스터 간 유사도 분석

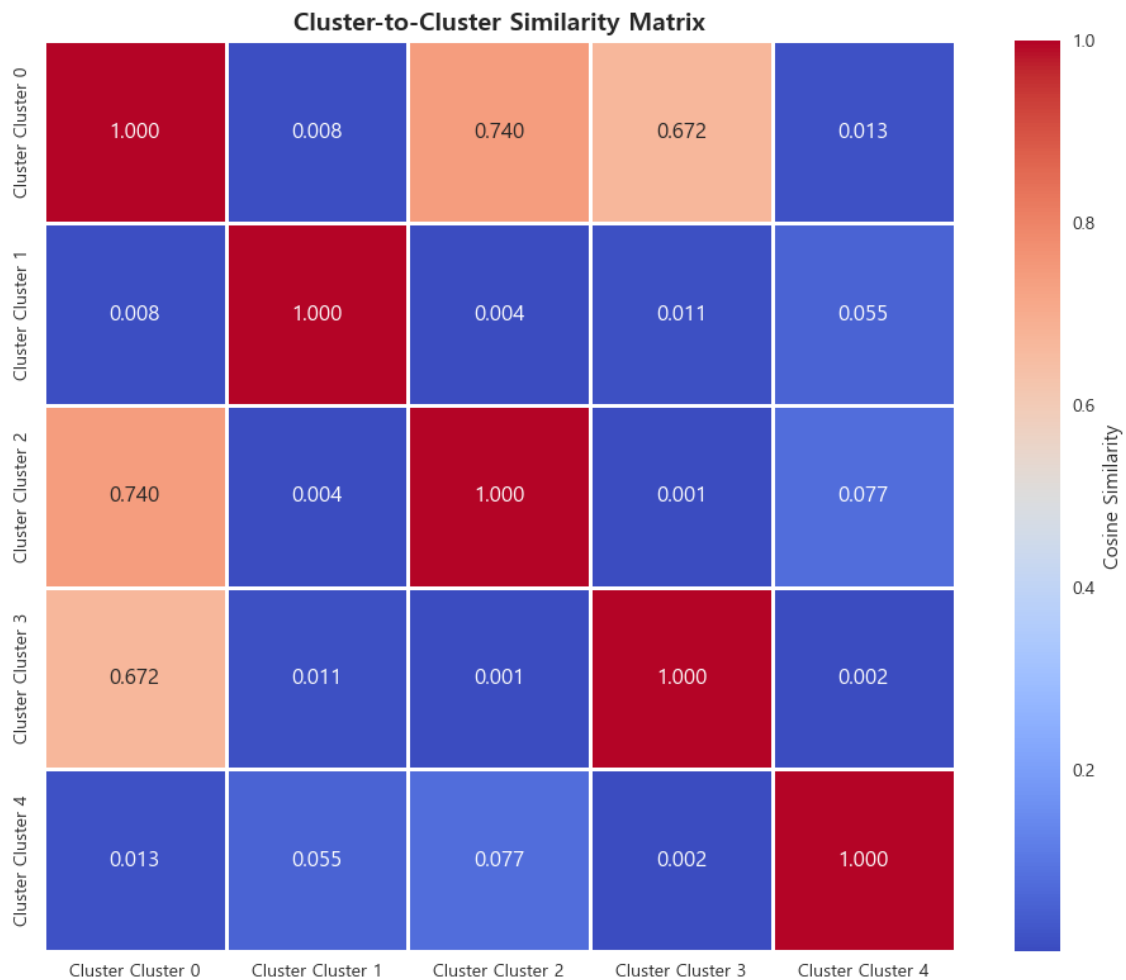
`analyze_cluster_similarity()`에서 각 클러스터에 속한 문서들의 문서-토픽 분포 평균을 클러스터 중심 벡터로 정의하고, 중심들 사이의 코사인 유사도로 클러스터-클러스터 유사도 행렬을 계산했다.

유사도 히트맵과 "가장 유사한 클러스터 쌍"을 출력함으로써, 서로 의미가 비슷해 병합을 검토할 수 있는 클러스터, 완전히 다른 주제를 다루는 클러스터를 구분할 수 있었다.

정리하면, 4.5.1에서는 LDA → 문서-토픽 분포 → 문서/클러스터 유사도라는 흐름으로 "주제 구조 관점의 유사도"를 시각적으로 탐색했다.



[그림 4.5.5 문서 간 유사도 히트맵]



[그림 4.5.6 클러스터 간 유사도 히트맵]

4.5.2 상세 유사도 분석 및 결과 저장

여기서는 LDA 토픽 공간이 아니라 TF-IDF 문서 벡터 공간에서 여러 군집 알고리즘(AP 포함)을 대상으로 정량적인 유사도 분석과 리포트 저장을 수행했다.

1. TF-IDF 기반 문서 벡터화 및 클러스터링 재적

Opinosis 원문(opinion_text)에 대해 TfidfVectorizer 로 최대 5,000 개의 단어에 대한 TF-IDF 행렬을 생성했다. PyCaret create_model / assign_model 을 활용해 K-Means, Birch, Agglomerative, Affinity Propagation(AP), DBSCAN, Spectral Clustering, OPTICS, MeanShift 등 여러 알고리즘에 대해 클러스터 레이블을

다시 부여했다. 이렇게 얻은 클러스터 레이블 + TF-IDF 벡터를 기반으로, 각 알고리즘의 군집 품질과 내부/외부 유사도를 세밀하게 비교했다.

2. 클러스터 내부/외부 유사도 측정 (코사인 + 유클리디안)

`analyze_cluster_similarity(X, labels, algorithm_name)` 함수에서 전체 문서에 대한 코사인 유사도 행렬을 먼저 계산하고, 각 클러스터별로 클러스터 내부(intra-cluster) 유사도를 산출했다. 평균, 표준편차, 최소/최대값을 보고 “응집력이 높고 고르게 뭉친 클러스터” vs “내부가 들쭉날쭉한 클러스터”를 구분했다. 동시에, 클러스터 중심(평균 TF-IDF 벡터)들 사이의 코사인 유사도로 클러스터 간(inter-cluster) 유사도 행렬을 만들어 가장 유사한 클러스터 쌍(병합 후보), 가장 구별되는 클러스터 쌍(주제적으로 완전히 다른 집단)을 도출했다.

`get_representative_docs()`에서는 각 클러스터의 중심과 개별 문서 사이의 유클리디안 거리(euclidean_distances)를 계산하고, 거리가 가장 짧은 문서들을 대표 문서(top-N)로 선정했다. 이 문서들의 파일명, 거리 값, 본문 미리보기를 함께 저장해 “클러스터를 대표하는 실제 예시 문서”를 쉽게 확인할 수 있게 했다.

◆ 클러스터 내부 유사도 (높을수록 클러스터가 응집력 있음)

```

Cluster 0 (크기: 3) - 평균: 0.8565, 표준편차: 0.0362, 범위: [0.8074, 0.8934]
Cluster 1 (크기: 3) - 평균: 0.6734, 표준편차: 0.1442, 범위: [0.5053, 0.8573]
Cluster 2 (크기: 4) - 평균: 0.1631, 표준편차: 0.1089, 범위: [0.0380, 0.3455]
Cluster 3 (크기: 3) - 평균: 0.2479, 표준편차: 0.1425, 범위: [0.0961, 0.4386]
Cluster 4 (크기: 2) - 평균: 0.4171, 표준편차: 0.0000, 범위: [0.4171, 0.4171]
Cluster 5 (크기: 2) - 평균: 0.2523, 표준편차: 0.0000, 범위: [0.2523, 0.2523]
Cluster 6 (크기: 2) - 평균: 0.8821, 표준편차: 0.0000, 범위: [0.8821, 0.8821]
Cluster 7 (크기: 3) - 평균: 0.3718, 표준편차: 0.4190, 범위: [0.0680, 0.9644]
Cluster 8 (크기: 2) - 평균: 0.9397, 표준편차: 0.0000, 범위: [0.9397, 0.9397]
Cluster 9 (크기: 3) - 평균: 0.5005, 표준편차: 0.1534, 범위: [0.3379, 0.7062]
Cluster 10 (크기: 3) - 평균: 0.2538, 표준편차: 0.1099, 범위: [0.1318, 0.3982]
Cluster 11 (크기: 2) - 평균: 0.6944, 표준편차: 0.0000, 범위: [0.6944, 0.6944]
Cluster 12 (크기: 4) - 평균: 0.6581, 표준편차: 0.2457, 범위: [0.3471, 0.9241]
Cluster 13 (크기: 5) - 평균: 0.4253, 표준편차: 0.2395, 범위: [0.1248, 0.8012]
Cluster 14 (크기: 5) - 평균: 0.3636, 표준편차: 0.3531, 범위: [0.1029, 0.9140]
Cluster 15 (크기: 3) - 평균: 0.2135, 표준편차: 0.1145, 범위: [0.0544, 0.3192]
Cluster 16 (크기: 2) - 평균: 0.9581, 표준편차: 0.0000, 범위: [0.9581, 0.9581]

```

◆ 클러스터 간 유사도 (낮을수록 클러스터가 잘 분리됨)

상위 5개 유사한 클러스터 쌍:

1. Cluster 12 ↔ Cluster 14: 0.3922
2. Cluster 9 ↔ Cluster 14: 0.3527
3. Cluster 9 ↔ Cluster 12: 0.3076
4. Cluster 6 ↔ Cluster 14: 0.3069
5. Cluster 4 ↔ Cluster 10: 0.3033

하위 5개 유사한 클러스터 쌍 (가장 구별됨):

1. Cluster 0 ↔ Cluster 16: 0.0064
2. Cluster 7 ↔ Cluster 16: 0.0076
3. Cluster 11 ↔ Cluster 16: 0.0083
4. Cluster 6 ↔ Cluster 7: 0.0089
5. Cluster 4 ↔ Cluster 6: 0.0089

3. 시각화: 클러스터 품질과 구조를 한눈에 확인

visualize_similarity() 함수는 하나의 그림 안에 네 가지 정보를 담았다.

- 클러스터별 평균 내부 코사인 유사도 바 차트

전체 평균 대비 각 클러스터가 얼마나 응집력이 높은지 비교.

- 클러스터 크기(문서 수) 분포

지나치게 큰/작은 클러스터가 존재하는지 확인.

- 클러스터 간 코사인 유사도 히트맵

색상으로 클러스터 간 유사도를 표현해, 군집 분리가 잘 되었는지 직관적으로 평가.

- 클러스터 품질 점수 바 차트

“품질 점수 = 내부 유사도 평균 - 외부(다른 클러스터와의) 평균 유사도”로 정의해 값이 클수록 “안에서 잘 뭉쳐 있고, 밖과는 잘 구분되는” 이상적인 클러스터로 해석했다.

각 알고리즘에 대해 위 네 가지 플롯을 생성하고,

../images/알고리즘명_similarity_타임스탬프.png 이름으로 저장했다.

Affinity Propagation 의 경우 별도로 클러스터 수(5, 6, 7)에 따른 지표를 비교한 문서에서 6 개 클러스터일 때 세 지표(Silhouette, Calinski-Harabasz, Davies-Bouldin)가 모두 가장 우수하다는 결론을 내렸고, AP 의 최적 클러스터 수를 6 으로 선택하였다.

4. 텍스트 리포트 저장

similarity_analysis_YYYYMMDD_HHMMSS.txt 파일에 아래 내용을 정리 저장했다.

- 알고리즘별 클러스터 내부 유사도 통계(평균, 표준편차, 범위, 클러스터 크기)
- 클러스터별 대표 문서 목록(파일명, 거리, 미리보기 텍스트)

이를 통해 코드 없이도 각 클러스터링 알고리즘의 특징과 대표 예시를 확인할 수 있게 했다.

정리하면, 4.5.2에서는 TF-IDF와 다양한 군집 알고리즘을 사용해 코사인 유사도 + 유클리디안 거리로 클러스터 품질과 대표 문서를 정량·정성적으로 분석하고, 그 결과를 이미지와 텍스트 리포트로 남겼다.

4.5.3 클러스터 유사도 시각적 결과

앞 절(4.5.2)에서 설명한 시각화 함수는 각 군집 알고리즘마다 “유사도 기반 품질 진단 대시보드”를 생성하는데, 이 중에서 Affinity Propagation(AP)에 대해 생성된 대표 결과가 다음 두 이미지이다.

[그림 4.5.7 AP 알고리즘 통계 (LDA)]



[그림 4.5.8 AP 알고리즘 통계 (NMF)]

1) 그림 구성

두 파일 모두 동일한 레이아웃을 사용한다.

- 좌상단: 클러스터별 평균 내부 코사인 유사도

- 우상단: 클러스터별 문서 수(클러스터 크기)
- 좌하단: 클러스터 간 코사인 유사도 히트맵
- 우하단: 클러스터 품질 점수(내부 유사도 - 외부 유사도) 수평 바 차트

이 한 장만 보면 어떤 클러스터가 응집력이 높고, 어떤 클러스터가 다른 군집과 비슷해서 병합 후보인지, 데이터가 특정 클러스터에 쏠려 있지는 않은지를 직관적으로 파악할 수 있다.

2) Affinity Propagation 결과 해석 요약

AP 는 이 프로젝트에서 최종 선택된 클러스터링 알고리즘으로, 사전 실험에서 Silhouette, Calinski-Harabasz, Davies-Bouldin 지표를 비교했을 때 클러스터 수 6 개일 때 군집 품질이 가장 우수한 것으로 확인되었다.

위 두 이미지는 AP 에 대해 서로 다른 설정/실행에서 얻은 클러스터 내부·외부 유사도 구조를 시각적으로 보여주며,

- 다수의 클러스터가 전체 평균보다 높은 내부 유사도를 가지는지,
- 특정 클러스터 쌍이 다른 쌍보다 유난히 높은 상호 유사도를 가지는지,
- 품질 점수가 낮은(= 재조정/병합이 필요한) 클러스터는 어디인지

를 확인하는 근거로 활용되었다.

따라서 4.5.3 에서는 Affinity Propagation 을 최종 모델로 선택한 뒤, 그 군집 구조가 실제로 "응집력 있고 잘 분리된 상태"인지를 시각적 결과(유사도 기반 대시보드 이미지)를 통해 검증한 과정을 정리한다.

4.6 결론 및 제언

4.6.1 주요 결과 요약

4.6.1.1 최종 선정 모델 및 성능

- 본 프로젝트 UCI Opinosis Opinion Review 실습 과정을 통해 LDA/NMF 기반 토픽 분포를 활용한 Affinity Propagation(AP) 클러스터링을 최종 모델로 채택하였다.
- AP 알고리즘은 클러스터 수를 자동 결정하며, Silhouette Score 0.9017, Davies-Bouldin Index 0.2581 를 기록하여 클러스터 간 명확한 분리와 응집도를 달성하였다.

Algorithm	N_Clusters	Silhouette	Davies-Bouldin
AP	5	0.9017	0.2581
SC	3	0.7707	0.4131
HCLUST	3	0.7707	0.4131
KMEANS	3	0.7602	0.5954
MEANSHIFT	3	0.6496	0.7794
DBSCAN	5	0.1669	1.4138

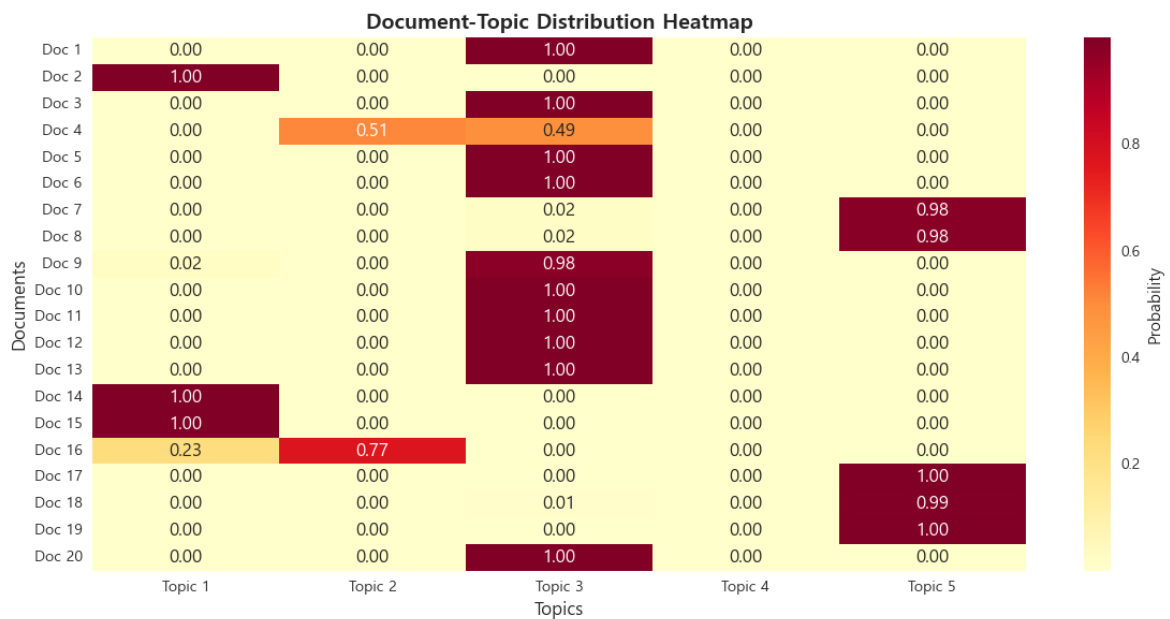
[표 4.6.1] LDA 기반 알고리즘별 클러스터 응집도/분리도 및 유사도 성능 비교 결과

참고: Silhouette Score: 높을수록 좋음, Davies-Bouldin: 낮을수록 좋음

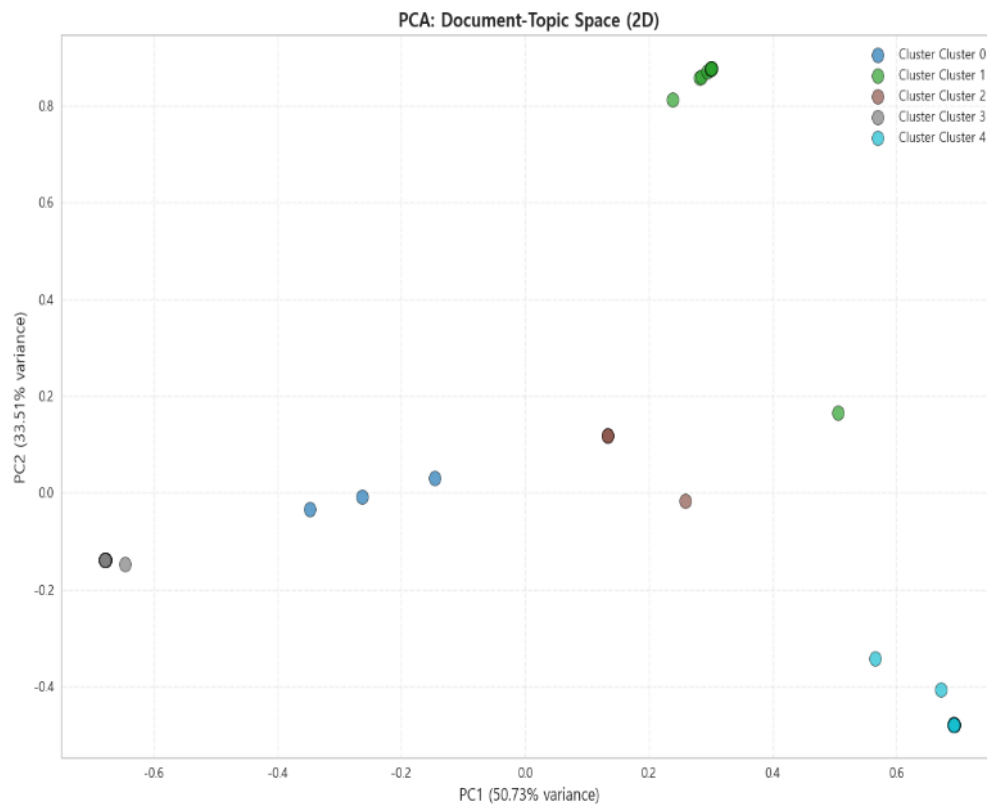
4.6.1.2 핵심 발견 사항

- 실습 과정을 통해 문서들이 밀접하게 연관된 5 개 클러스터로 명확히 분리됨을 확인되었으며, 각 클러스터별 대표 키워드 및 문맥적 특징을 바탕으로, 리뷰 성향, 주제별 논점, 사용자 관심사 등에서 의미 있는 인사이트를 도출할 수 있었다.
 - LDA 토픽 모델링: 리뷰 데이터에서 호텔, 전자기기, 자동차 등 5 개 주요 토픽 도출
 - 문서별 토픽 분포가 뚜렷하게 나타나며 의미 있는 주제 분류 가능

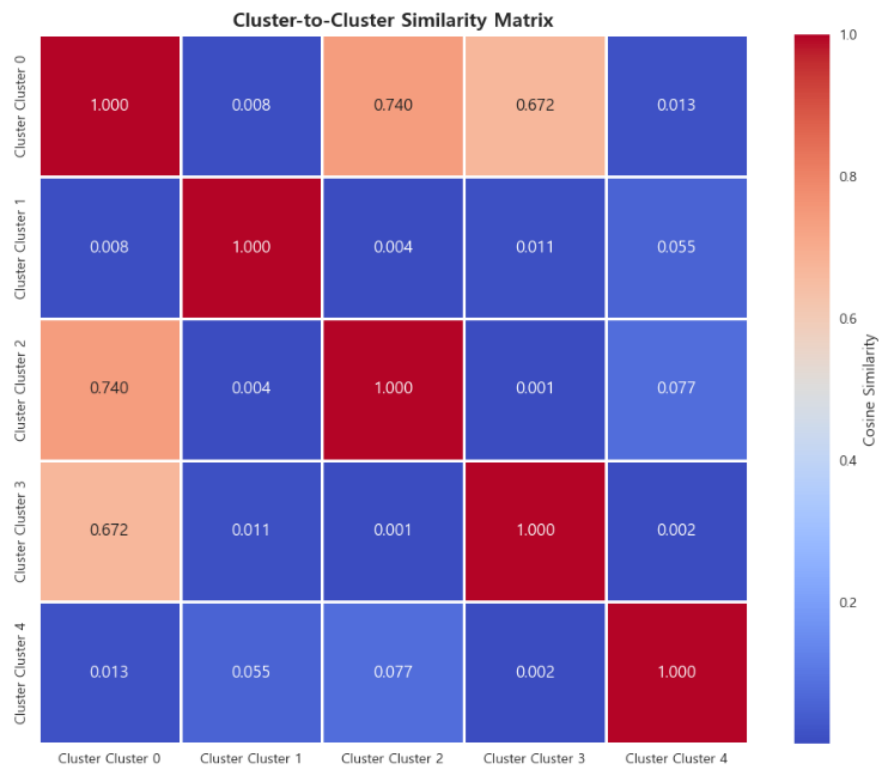
- PyCaret 기반 기본 벡터라이징보다 LDA/NMF 토픽 모델링 적용 시 군집화 품질과 해석력이 유의미하게 향상됨을 확인하였다.
- 시각화 및 문서 유사도 분석에서는 총 설명된 분산 84.24% 로 차원 축소 후에도 데이터의 변동성을 상당히 잘 설명하고 있었다.
- 문서 유사도 결과, Top5 유사 문서 모두 유사도 1.0 으로 벡터화된 표현에서 거의 동일한 의미 공간에 위치하였고, 유사 문서들은 소프트웨어 설치, 디스플레이, 하드웨어 성능, 가격 비교 등 제품 사용 경험과 관련된 리뷰들로, 같은 클러스터 내에서 강한 연관성을 보여주었다.
 - 시각화 및 유사도 분석을 통해 동일 클러스터 내 문서들이 실제 의미적으로도 높은 유사성을 보여 결과의 타당성을 확인하였다.



[그림 4.6.1] 문서별 토픽 분포 Heatmap 시각화



[그림 4.6.2] PCA 를 이용한 2D 토픽 공간 시각화



[그림 4.6.3] 문서 유사도 행렬 Heatmap 시각화

4.6.2.1 데이터 측면 한계점

- 텍스트 데이터의 정제 품질에 따라 토픽 모델링 결과가 민감하게 변화하였으며, 불용어 처리·중복 데이터·비표준 표현(오타, 속어 등)이 성능에 영향을 미쳤다.
- 데이터 양이 방대할 경우 토픽 모델링 및 Affinity Propagation(AP) 클러스터링의 처리 시간이 상당히 증가하여, 대규모 데이터셋 적용 시 효율성에 제약이 있었다.
- 데이터셋이 특정 도메인(호텔, 전자기기, 자동차 등)에 집중되어 있어, 다른 분야 리뷰에 대한 일반화 가능성은 제한적이었다.
- 비지도 학습 기반 데이터 특성상 라벨이 부재하여, 결과 검증 시 외부 기준이나 전문가 해석이 필요하였다.

4.6.2.2 모델 측면 한계점

- LDA/NMF의 토픽 개수 결정은 휴리스틱에 의존하여 주관적 요소가 개입될 수 있었다.
- AP 클러스터링은 초기 선택 및 하이퍼파라미터 설정에 민감하여 결과 재현성 확보에 한계가 있었다.
- Silhouette Score와 Davies-Bouldin Index만으로 클러스터 품질을 평가할 경우, 실제 문서 맥락과의 비교 분석이 부족하였다.
- 토픽 모델링 결과에서 키워드 중첩이나 의미 모호성이 발생하여, 토픽 해석 과정에서 주관적 판단이 불가피하였다.
- 리뷰 데이터 벡터화 과정에서 차원이 매우 커져(수천~수만 축) 클러스터링 알고리즘의 계산 복잡도가 증가하고, 시각화 및 해석에 제약이 있었다.
- 현재 접근법은 정적 데이터셋에 최적화되어 있어, 신규 문서가 지속적으로 유입되는 환경에서는 실시간 업데이트 및 확장성 확보가 어려웠다.

4.6.3 개선 방안 및 향후 계획

4.6.3.1 토픽 모델링 고도화

- BERT 기반 토픽 모델(BERTopic) 등을 활용하여 토픽 품질 및 의미론적 해석을 강화할 필요가 있다.
- Word2Vec, Sentence-BERT 등의 임베딩을 활용한 의미 기반 군집화를 적용하여, 단순 빈도 기반 분석을 넘어 문맥적 유사성을 반영할 수 있다.
- GraphRAG와 같은 최신 그래프 기반 접근법을 도입하여 단어 간 관계성을 구조적으로 반영함으로써, 기존 모델의 한계를 보완할 수 있다.

4.6.3.2 평가 지표 다각화

- Silhouette Score, Davies-Bouldin Index 외에도 Coherence Score 등 토픽 모델링 자체 평가 지표를 추가하여 클러스터 및 토픽 품질을 종합적으로 평가
- 클러스터별 실제 문서 비교 및 정성적 평가를 병행하여, 수치적 지표와 의미론적 타당성 동시 확보
- 사용자 피드백 기반 평가를 도입하여, 모델 결과가 실제 활용 맥락에서 유용한지 검증 요구

4.6.3.3 실무 적용 방안

- 각 클러스터를 자동 레이블링하는 시스템 구축으로, 리뷰 데이터 관리 및 분석 효율성 제고
- 신규 문서 유입 시, 유사 문서 자동 추천 시스템으로 확장하여, 실시간 데이터 처리 및 활용성 강화
- 마케팅, 제품 전략 수립, UX 개선 등 실무 의사결정 지원에 직접 활용할 수 있도록, 분석 결과를 대시보드 형태로 시각화하는 방안을 고려
- 지속적인 데이터 업데이트와 모델 재학습 체계를 마련하여, 변화하는 사용자 리뷰 환경에 적응할 수 있는 확장성 확보

4.6.4 실습 정리 및 요약

이번 프로젝트에서는 UCI Opinions Opinion Review 데이터셋을 대상으로 다양한 클러스터링 알고리즘을 비교 분석하였으며, 특히 Affinity Propagation(AP) 알고리즘이 가장 우수한 성능을 나타냈다. AP는 클러스터 수 5개를 자동 결정하며 Silhouette Score 0.9017, Davies-Bouldin Index 0.2581를 기록하여 클러스터 간 분리와 응집도가 뛰어남을 확인했다. K-Means, Spectral Clustering, Hierarchical Clustering 등도 준수한 성능을 보였으나, DBSCAN은 Noise 데이터에 민감하여 상대적으로 낮은 점수를 기록했다.

회의를 통해 팀에서는 PyCaret 기반 벡터라이징 방식보다 LDA 토픽 모델링이 클러스터링 성능 향상에 유의미함을 확인했다. 최종적으로 토픽 5개를 추출하고, AP 알고리즘을 활용하여 시각화 및 유사도 분석을 수행하였으며, 군집별 상위 키워드를 통해 결과의 의미를 검증했다.

4.6.4.1 시사점 및 제언

1) 실무적 활용

- 리뷰 데이터를 토픽별로 자동 분류하고 핵심 의견을 추출하여 고객 피드백 분석, 제품 개선 전략 수립 등에 활용 가능.
- AP 알고리즘은 클러스터 수를 사전에 지정할 필요가 없어, 데이터 특성이 불확실한 상황에서도 유연하게 적용 가능.

2) 데이터 기반 전략

- 각 클러스터의 대표 키워드와 리뷰 특징을 분석하여 사용자 경험(UX) 개선 포인트 도출 가능.
- 클러스터링 결과와 실제 문서 제목, 리뷰 내용과 비교 분석함으로써 모델 성능 검증 및 개선 가능.

3) 연구 및 분석 확장

- Silhouette 점수만을 신뢰하기보다는 클러스터별 실제 데이터 비교를 통한 정성적 평가 필요.

- TF-IDF 외 Word2Vec, Sentence-BERT 등의 임베딩 기반 벡터화를 적용하면 의미 기반 유사도 분석 정확도 향상 가능.
- 장기적으로 리뷰 변화 추이를 반영한 시계열 군집 분석이나 Ensemble Clustering 시도도 고려 가능

이번 분석을 통해, 단순한 군집화 지표 평가를 넘어 토픽 해석, 키워드 분석, 시각화 및 실제 데이터 검증을 함께 수행하는 것이 리뷰 데이터 분석에서 핵심임을 확인할 수 있었다.

4.7 참고자료 및 부록

4.7.1 참고 자료

4.7.1.1 참고 문헌

UCI Machine Learning Repository: Opinion Review 데이터셋을 다운로드하고 프로젝트를 시작하는 데 사용된 공식 데이터 출처
(<https://archive.ics.uci.edu/dataset/191/opinosis+opinion+frasl+review>).

scikit-learn Documentation: 텍스트 전처리(CountVectorizer, TfidfVectorizer), 토픽 모델링(LDA, NMF), 기본 군집화 알고리즘(K-Means, Spectral Clustering 등), 차원 축소(PCA), 그리고 유사도 계산(코사인 유사도, 유클리디안 거리) 구현 및 활용에 참고되었습니다.

PyCaret Documentation (pycaret.clustering): 다수의 군집화 알고리즘(Affinity Propagation, K-Means, DBSCAN 등)을 통일된 평가 지표(Silhouette, CH, DB Index) 기준으로 자동 비교 및 최적 모델 선정 과정을 구현하는 데 핵심적으로 활용되었습니다.

NLTK (Natural Language Toolkit) Documentation: 토큰화, 불용어 제거, 표제어 추출(Lemmatization) 등 텍스트 전처리 로직(TextPreprocessor 클래스) 설계 및 구현에 활용되었습니다.

matplotlib, seaborn, plotly Documentation: 군집 결과(Elbow Method, Silhouette Score), 토픽 분포, 문서 간 유사도 히트맵, PCA 기반 2D 군집 구조 등 모든 시각화 결과물 생성 및 한글 폰트 설정에 참고되었습니다.

4.7.1.2 사용 라이브러리 버전

Python=3.10.19 기본 실행 환경

Pycaret=3.3.2 주력 ML/DS 자동화 도구

Pandas=2.1.4 데이터 처리 및 분석

Numpy=1.26.4 고성능 수치 계산

scikit-learn=1.4.2 기본 머신러닝 알고리즘 및 유틸리티

nltk=3.9.2 자연어 처리 및 텍스트 전처리

matplotlib=3.7.5 데이터 시각화

seaborn=0.13.2 통계 시각화

plotly=5.24.1 인터랙티브 시각화

umap-learn=0.5.7 차원 축소 및 시각화 (UMAP)

scipy=1.11.4 과학 계산 및 통계

4.7.2 부록

4.7.2.1 코드 저장소 및 주요 파일 설명:

코드저장소: <https://github.com/jaemin1020/big20-ML-project2-team3>

디렉토리	포함 파일 및 내용	보고서 내 역할 (출처)
/config	pycaret_env.yml, pycaret_setup.md	실험 환경 관리
/data	Opinion Review 원본 텍스트 파일 (51 개)	원본 데이터
/utils	preprocessing.py (또는 utils.py)	공통 유틸리티
/src	topic_modeling_lda.ipynb	토픽 모델링
	clustering+similarity_vis_final, NMF_clustering+similarity_vis	유사도 분석 및 시각화
/images	그림 4.5.1 문서 토픽 분포 히트맵.png 등	시각화 결과물
/results	similarity_analysis_*.txt, cluster_results_ap.csv 등	최종 분석 결과

4.7.2.2 실행 환경 구축 및 모델 재현 방법:

```
conda env create -f pycaret_env.yaml  
conda activate pycaret_env
```

NLTK 데이터 다운로드: 자연어 처리에 필요한 NLTK 데이터를 다운로드하였다.

```
import nltk  
nltk.download("punkt")  
nltk.download("wordnet")
```

코드 실행: 환경이 성공적으로 활성화된 후, 프로젝트의 Jupyter Notebook 파일들을 순서대로 실행하여 모든 모델 학습 및 분석 결과를 재현할 수 있습니다. 사용된 Python 버전은 3.10.19이며, 이 환경 내에서 모든 종속성이 관리됩니다.